

Reflective Journal

NewsBot Intelligence System 2.0 — ITAI 2373

Submitted by: Nertnapha Sanwivat | Group: Individual Project

1. Individual Contributions

As the sole member of this project team, I was responsible for the full lifecycle of NewsBot 2.0: architecture planning, implementation of all eight core NLP components, system integration, testing, evaluation, and documentation.

- Designed the modular system architecture, defining NewsBot2Config and NewsBot2System as the foundation for every downstream component.
- Built the Content Analysis Engine: a TF-IDF + Logistic Regression classifier with confidence scoring and explainability, an LDA-based topic discovery engine, a DistilBERT-based sentiment tracker with temporal analysis, and a spaCy/NetworkX entity relationship mapper.
- Built the Language Understanding & Generation layer: a BART-based abstractive summarizer, a sentence-transformer semantic search engine, and a content enhancement module for surfacing coverage gaps.
- Built the Multilingual Intelligence layer, including language detection, translation, and cross-lingual comparison across English, Spanish, and French.
- Built the Conversational Interface, including intent classification, query-entity extraction, and natural-language response generation.
- Integrated every component into NewsBot2IntegratedSystem with isolated error handling, and validated the result with a full test suite (NewsBot2TestSuite) and evaluation framework (NewsBot2Evaluator).
- Authored the technical documentation, executive summary, and presentation deck for the final deliverable.

2. Challenges Overcome

Balancing model size against inference speed

Several components rely on pretrained transformer models — DistilBERT for sentiment, BART for summarization, and MiniLM-based sentence transformers for semantic search. Running all of these together, on limited local compute, meant I had to be deliberate about which models to load and when. I addressed this by choosing distilled/smaller variants where possible (DistilBERT over full BERT, MiniLM over larger sentence-transformer models) and by isolating each component so that a slow or failed model load never blocked the rest of the pipeline.

Handling ambiguous, multi-part conversational queries

Getting the Conversational Interface to reliably classify intent and pull out the right entities from a loosely phrased question (for example, 'How does coverage of the election differ by region?') was harder than I expected. Queries often combine more than one intent, or reference something from an earlier turn in the

conversation. I worked through this by separating intent classification from entity extraction into distinct steps, and by building a simple conversation-history mechanism so follow-up questions could be resolved using prior context rather than treated as entirely new queries.

Maintaining topic coherence on short, mixed-topic articles

LDA topic modeling performed well on longer, single-theme articles (business and politics coverage in particular), but coherence dropped noticeably on shorter articles that blended multiple themes in a few sentences. This was a useful reminder that a technique can work very well on the data it was tuned against and still need adjustment for edge cases in the real distribution of inputs.

3. Lessons Learned

- Fallback handling is what actually keeps a multi-component system usable. Wrapping each component call in isolated error handling (the `_safe_run` pattern in `NewsBot2IntegratedSystem`) meant that a single model failure degraded one feature instead of taking down the whole analysis pipeline — a lesson I will carry into any future system with multiple independent AI components.
- Starting from strong pretrained transformers (BART, DistilBERT, sentence-transformers) dramatically cut development time compared to training models from scratch, and let me focus engineering effort on integration, orchestration, and the conversational layer rather than on basic model training.
- Evaluating early and often — rather than saving testing for the end — surfaced weaknesses (like topic coherence on short articles) while there was still time to understand and document them properly, instead of discovering them at the last minute.
- Designing for modularity from the start (separate classes per capability, coordinated by a single orchestrator) made it far easier to test, debug, and reason about each piece of the system independently.

4. Future Improvements

Looking ahead to a NewsBot 3.0, the improvements I would prioritize are:

- Fine-tuning transformer models (BERT/RoBERTa) specifically on news classification data, rather than relying solely on general-purpose pretrained weights, to push classification accuracy higher.
- Adding real-time streaming ingestion so the system can process live news feeds as they publish, instead of running in scheduled batches.
- Building bias and fact-check detection to flag potential political or cultural bias and cross-reference claims, which would meaningfully increase the trustworthiness of automated insights.
- Developing a web application front end so non-technical stakeholders can explore NewsBot's insights directly, without needing to run the notebook.

Overall, this project pushed me to combine nearly everything covered in this course — classification, topic modeling, sentiment analysis, named entity recognition, summarization, semantic search, multilingual NLP, and conversational query handling — into a single working system, and to think about it not just as a set of separate techniques but as a product with real users, real failure modes, and real trade-offs.